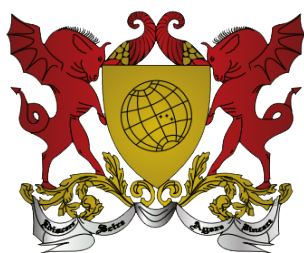


Aprendizado Profundo

Pedro Sérgio Tôrres Figueiredo Silva



Universidade Federal de Viçosa

Reitor

Demetrius David da Silva

Vice-Reitora

Rejane Nascentes



Diretor

Francisco de Assis de Carvalho Pinto

Campus Universitário, 36570-900, Viçosa/MG

Telefone: (31) 3612 1251

Layout: Ana Luísa Medeiros e Stéfany Peron

Editoração Eletrônica: Hugo Virgínio

Edição de conteúdo e CopyDesk: João Batista Mota



Significado dos ícones da apostila

Para facilitar o seu estudo e a compreensão imediata do conteúdo apresentado, ao longo de todas as apostilas, você vai encontrar essas pequenas figuras ao lado do texto. Elas têm o objetivo de chamar a sua atenção para determinados trechos do conteúdo, com uma função específica, como apresentamos a seguir.



DESTAQUE: são definições, conceitos ou afirmações importantes às quais você deve estar atento.



GLOSSÁRIO: Informações pertinente ao texto, para situá-lo melhor sobre determinado termo, autor, entidade, fato ou época, que você pode desconhecer.

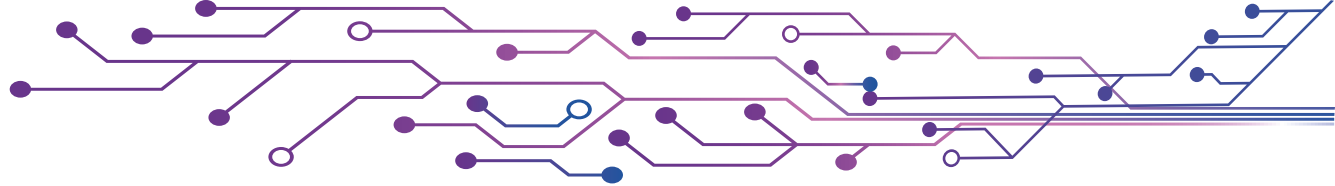
SAIBA MAIS: se você quiser complementar ou aprofundar o conteúdo apresentado na apostila, tem a opção de links na internet, onde pode obter vídeos, sites ou artigos relacionados ao tema.



PARA REFLETIR: vai fazer você relacionar um tópico a uma situação externa, em outro contexto



Exercícios Propostos: são momentos pra você colocar em prática o que foi aprendido



PEDRO SÉRGIO TÔRRES FIGUEIREDO SILVA

É graduado em Engenharia Mecatrônica (2015) e tem mestrado em Engenharia Elétrica (2018) pela Universidade Federal de São João Del-Rei (UFSJ). Atualmente, é doutorando do programa de Engenharia Elétrica da Universidade Federal do Rio de Janeiro (UFRJ).

Desde 2020, trabalha como cientista de dados na empresa Ambevtech, desenvolvendo análises e soluções com dados dentro do contexto de People Analytics. Atua em pesquisas relacionadas à aplicação de aprendizado de máquina e processamento de sinais, principalmente em problemas de engenharia biomédica.

Contato: pedrosergiot@lps.ufrj.br



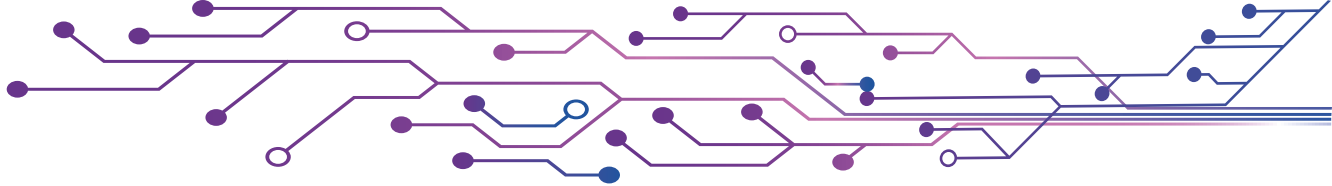
Apresentação

Esta apostila tem como objetivos apresentar os principais conceitos do Aprendizado Profundo de uma maneira objetiva e prática. Neste material serão abordadas as redes neurais profundas, convolucionais, recorrentes e técnicas para treinamentos desses modelos.

Com os conceitos e explicações apresentados aqui, juntamente com o conteúdo das aulas práticas e atividades, você terá autonomia para aplicação dos métodos estudados em outros problemas.

Dedique-se às atividades e tire as suas dúvidas durante as aulas para ter o melhor aproveitamento possível do curso.

Bons estudos!



Sumário

1. CONCEITOS DE APRENDIZADO PROFUNDO E TREINAMENTO DE REDES NEURAIS PROFUNDAS	9
1. APRENDIZADO PROFUNDO	9
2. PROBLEMA DA DISSIPAÇÃO DOS GRADIENTES / GRADIENTES EXPLOSIVOS	10
3. PROBLEMA DA VELOCIDADE DE TREINAMENTO	12
4. PROBLEMA DO SOBREAJUSTE	13
2. REDES NEURAIS CONVOLUCIONAIS	16
1. O CÓRTEX VISUAL	16
2. A CAMADA CONVOLUCIONAL	16
3. A CAMADA <i>POOLING</i>	18
4. <i>PADDING</i> E <i>STRIDE</i>	19
3. REDES NEURAIS RECORRENTES	20
1. OS NEURÔNIOS RECORRENTES	20
2. SEQUÊNCIAS DE ENTRADAS E SAÍDAS	21
3. A CÉLULA LSTM	22
4. AUTOENCODERS E TRANSFER LEARNING	23
1. O AUTOENCODER	23
2. TRANSFER LEARNING	25



1. Conceitos de Aprendizado Profundo e Treinamento de Redes Neurais Profundas

Para vários problemas atuais, como classificação de imagens e reconhecimento por voz, as técnicas mais tradicionais de aprendizado de máquina geralmente não obtêm bons resultados, o que se deve muito ao grau de complexidade dessas tarefas. A área denominada aprendizado profundo (*Deep Learning*) surgiu como um ramo dentro do tópico aprendizado de máquina (*Machine Learning*) para resolver tais problemas complexos, usando como base o modelo de redes neurais artificiais (RNA) e propondo modificações deste tipo de modelo para tarefas específicas, como as redes neurais convolucionais (RNC) e redes neurais recorrentes (RNR).

Nesta unidade abordaremos os principais conceitos envolvendo o aprendizado profundo e também faremos uma breve visita ao tópico do aprendizado não-supervisionado.

1. APRENDIZADO PROFUNDO

O aprendizado profundo moderno é a evolução dos modelos de aprendizado de máquina mais simples, sendo inspirado na biologia para imitar/emular o modo como nós, seres humanos, adquirimos conhecimento. O modelo das redes neurais artificiais foi o primeiro passo para tentar alcançar tal objetivo, com a maior parte dos modelos modernos de aprendizado profundo baseados em RNNs. A diferença fica por conta da profundidade/complexidade desses modelos, usando um grande número de camadas para extrair a informação de forma progressiva a partir dos dados de entrada.

Redes neurais profundas (também chamadas de *Percepção de Múltiplas Camadas - MLP*) com 1 ou duas camadas ocultas são utilizadas para resolver uma vasta gama de problemas (de fato, o *Teorema da Aproximação Universal* diz que uma rede neural com 1 camada oculta pode aproximar qualquer função contínua dentro de uma faixa específica). Isso geralmente é o suficiente para muitos problemas envolvendo classificação ou regressão de dados mais simples, como aqueles advindos de tabelas.



Intuitivamente, isso pode nos levar a pensar que para problemas mais complexos, envolvendo dados como imagens ou áudios, basta adicionar mais camadas à rede neural e ela será capaz de aprender uma solução. No entanto, encontramos alguns problemas para treinar uma rede muito grande, com um grande número de conexões:

- Primeiro, teríamos o problema da dissipação do gradiente (*Vanishing Gradient*) ou do gradiente explosivo (*Explosive Gradiante*), que iremos explorar com mais detalhes à frente e que tornam o treinamento de redes neurais profundas bem difícil para as camadas inferiores;
- Quanto maior o tamanho da rede, maior também será o tempo de treinamento e os requisitos de capacidade computacional para executar tal treinamento;
- Como uma rede neural tem muitos parâmetros (pesos) para serem ajustados, aumenta-se o risco de ocorrer sobreajuste do conjunto de treinamento, especialmente para bancos de dados pequenos.

Apesar de existirem técnicas para mitigar esses três problemas (como veremos neste capítulo) ainda existe espaço para que sejam introduzidas outras arquiteturas de redes para tarefas específicas, o que acaba se alinhando com a inspiração biológica que está por trás das redes neurais (os processos do corpo humano que extraem a informação de interesse de uma imagem ou do som são diferentes, sendo específicos para cada tarefa). Neste ponto, adentramos no tópico de arquiteturas como redes neurais convolucionais e redes neurais recorrentes, que são redes neurais profundas com modificações em suas arquiteturas para aplicação em tarefas específicas.

Graças aos modelos profundos, hoje temos soluções para diversos problemas nos campos de visão computacional, processamento de áudio, processamento de linguagem natural (*Natural Language Processing* - NLP) e vários outros. Tarefas complexas, como a direção de piloto automático em carros, classificação de avaliação de usuário, sistemas de recomendação e previsão de séries temporais meteorológicas, são hoje resolvidas por esses modelos. Eles ainda são bastante estudados e aplicados em empresas como o Google, Meta, Amazon e outras que têm uma grande quantidade de dados (o que quase sempre é o principal requisito para o desenvolvimento desse tipo de modelo).

2. PROBLEMA DA DISSIPAÇÃO DOS GRADIENTES / GRADIENTES EXPLOSIVOS

Para entender melhor como esses dois problemas afetam o treinamento de redes neurais com muitas camadas, tornando mais difícil o emprego delas, relembremos brevemente como ocorre o treinamento de um modelo de rede neural:

- Na primeira fase, chamada *forward*, os dados são apresentados às entradas da rede, sendo propagados até o final dela e nos dando um resultado/saída para cada amostra/observação usada.
- As saídas obtidas pela rede são comparadas com os valores esperados para cada amostra, sendo gerado um valor de erro entre a saída que a rede apresentou e a que ela deveria apresentar.
- Os erros individuais são agregados e retropropagados da saída da rede até sua entrada. Essa retropropagação ocorre de camada para camada da rede, sendo calculados os **gradientes** em cada passo e usados para atualizar os pesos daquela camada (uso do Gradiente Descendente).
- Os passos de 1 a 3 se repetem até que o erro seja zerado, chegue em um patamar aceitável ou que se atinja algum outro critério de parada de treinamento.

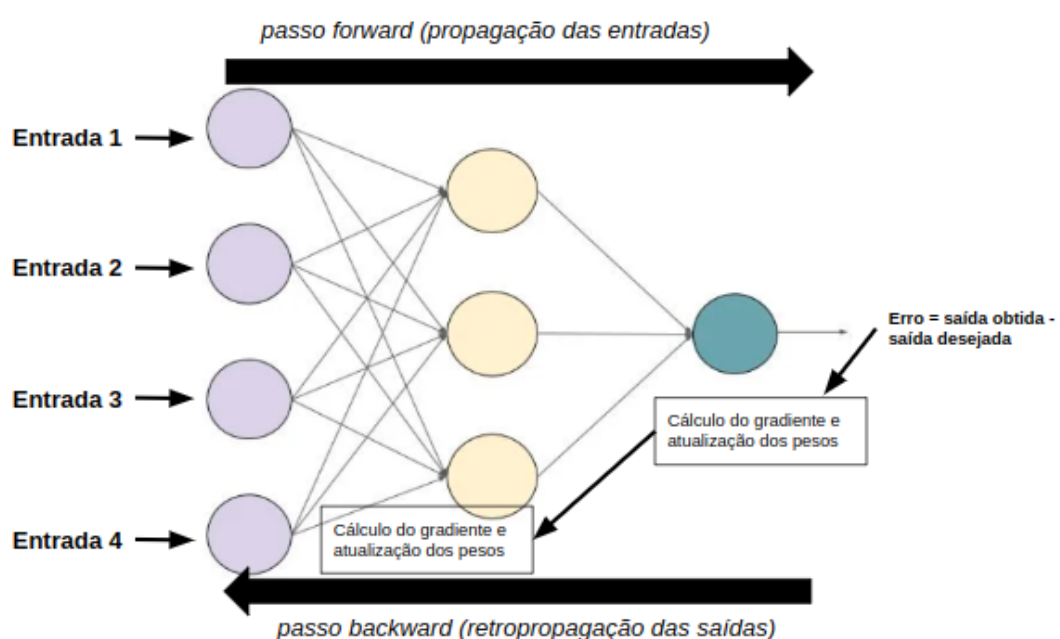


Figura 1: Passos forward and backward do treinamento e uma RNA

O *problema dos gradientes dissipados* ocorre devido aos gradientes calculados a cada camada irem diminuindo à medida que o algoritmo avança para camadas inferiores. Com poucas camadas, esse efeito tende a ser minimizado, mas para uma rede com muitas camadas ocorrerá de os gradientes das primeiras camadas serem tão pequenos que praticamente deixarão os pesos dessas camadas inalterados quando usados para atualizá-los.



Assim, o treinamento não consegue convergir para uma boa solução pois as primeiras camadas não estão sendo alteradas durante o treino do modelo. Algo semelhante ocorre para o *problema dos gradientes explosivos*: nesse caso a propagação dos gradientes faz com que eles cresçam para valores muito elevados e as atualizações dos pesos ficam intensamente grandes, fazendo a rede divergir e nunca chegar a uma solução.

Os dois problemas descritos ficaram um bom tempo sem que cientistas tivessem uma boa compreensão sobre eles, até que em 2010 foram feitas algumas contribuições em um trabalho publicado por Xavier Glorot e Yoshua Bengio que levaram às técnicas que vamos discutir a seguir.

2.1 Cuidado com a Inicialização Usada

Provavelmente, a técnica de inicialização mais utilizada seja a de usar pequenos valores aleatórios dentro de um intervalo como -1 e 1. No entanto, essa inicialização pode fazer com que o sinal não flua corretamente na etapa de propagação (*forward*) ou na de retropropagação (*backward*). Isso pode resultar no sinal desaparecendo ou explodindo em uma dessas fases. Para atender, ao menos parcialmente, esse requisito, podem ser utilizados dois tipos de inicialização específicas:

- **Inicialização Xavier ou Inicialização Glorot:** os pesos de uma camada devem ser inicializados levando-se em conta o número de entradas e saídas daquela camada. Isso é feito com valores aleatórios vindos de uma distribuição normal com média 0 e desvio padrão dado pela fórmula a seguir.

$$\sigma = \sqrt{\frac{2}{n_{\text{entradas}} + n_{\text{saídas}}}}$$

Essa inicialização acelera bastante o treino do modelo e é um dos pontos responsáveis pelo atual sucesso de modelos profundos.

- **Inicialização He:** também considera o número de entradas e saídas de uma camada da rede para inicializar os pesos dela, mas faz isso usando fórmulas específicas para o desvio padrão, considerando o uso de uma função de ativação ELU ou ReLU (que discutiremos a seguir).

Os principais pacotes para desenvolvimento de treinamento de redes neurais já incluem essas e outras inicializações, sendo necessário apenas escolher o tipo que será usado (comumente, a inicialização Xavier/Glorot é usada como padrão).

2.2 Função de Ativação

Talvez a função de ativação mais comumente empregada em redes mais rasas seja a sigmoide ou a tangente hiperbólica, que realmente funcionam e fazem parte dos fundamentos desses modelos. No entanto, para modelos mais profundos, o uso dessas funções saturadas pode causar os problemas de gradiente.

Assim, recomenda-se o uso de funções não saturadas para modelos profundos, principalmente a função LeakyReLU, já que esta não tem região de saturação (região a partir da qual a função alcança um valor constante ou quase constante) e soluciona alguns problemas da sua função irmã ReLU. O parâmetro de vazamento a da LeakyReLU pode ser ajustado e geralmente fica entre os valores 0.2 e 0.01

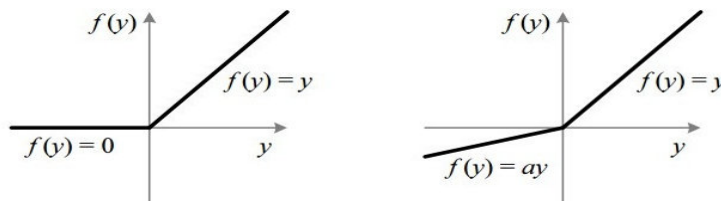


Figura 2: Funções ReLU e LeakyReLU.

2.3 Normalização em Lote

Talvez esta seja a técnica mais popular empregada em modelos profundos. Embora a combinação de inicialização Xavier e função de ativação não saturada possa prevenir os gradientes explosivos ou dissipados no início do treinamento, eles ainda podem voltar durante o treinamento. É aqui que entra a normalização por lote, sendo introduzida por Sergey Ioffe e Christian Szegedy em trabalho publicado no ano de 2015.

Aqui devemos lembrar que muitas vezes o treinamento é feito por lotes de amostras/observações dos dados, já que muitas vezes todo o conjunto de treinamento pode não caber na memória disponível. A normalização por lote consiste em adicionar uma operação antes da função de ativação de cada neurônio, consistindo de uma normalização das saídas de cada amostra do lote.



Essa técnica tem várias vantagens, entre elas a redução da ocorrência dos problemas relacionados aos gradientes de modo que seja possível até usar funções sigmoide como ativação da rede. As únicas desvantagens desta técnica ficam por conta de um aumento no tempo de execução do treinamento e a adição de um pouco mais de complexidade ao modelo.

Novamente, os pacotes mais populares para treinamento de modelos profundos, como o TensorFlow e o PyTorch, já possuem esse tipo de normalização implementada, restando ao usuário a escolha de adicioná-la ou não ao seu modelo.

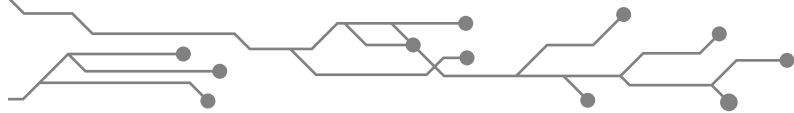
3. PROBLEMA DA VELOCIDADE DE TREINAMENTO

Talvez um dos maiores fatores de impedimento para o treinamento de modelos profundos seja o tempo de treinamento: como existem muitos parâmetros para serem ajustados e muitos dados a serem transitados nas etapas de treinamento, o tempo para o treinamento do modelo cresce consideravelmente. Assim, foram desenvolvidas algumas técnicas que podem ajudar a reduzir o tempo de treinamento do modelo.

3.1. Reutilizar Camadas Pré-Treinadas

Essa técnica bem popular, chamada de *transfer learning*, pode economizar uma grande quantidade de tempo durante o treinamento de um modelo profundo. Ela se baseia na utilização das primeiras camadas de um modelo já treinado para outra tarefa similar à que temos em mãos. Assim, basicamente cortamos as últimas camadas deste modelo pré-treinado e adicionamos novas camadas que são treinadas para realizar a nova tarefa, congelando as atualizações das camadas pré-treinadas (após o primeiro treinamento, podemos fazer um ajuste fino dessas camadas ao liberar a atualização de seus pesos numa segunda rodada de treinamento).

A ideia por trás do *transfer learning* vem de que as primeiras camadas de uma rede serão responsáveis por extrair características dos dados, sendo que camadas mais perto da saída da rede extraem características de ordem mais alta. As últimas camadas da rede fazem a combinação dessas características para realizar a tarefa de classificação/regressão requerida. Assim, se já temos um modelo treinado capaz de extrair as características de interesse dos dados, temos apenas que treinar as camadas que combinam essas características extraídas de outros dados para a realização de outra tarefa de classificação/regressão (um exemplo seria usar uma rede pré-



treinada para classificação de animais para compor uma rede que será treinada para classificar veículos).

Na última semana do curso, abordaremos o *transfer learning* em uma atividade prática relacionada aos outros modelos que veremos nas semanas 2 e 3.

3.2. Otimizadores Mais Velozes

O otimizador é o algoritmo usado para tentar minimizar o erro obtido pelo modelo a cada passo de treinamento. Os otimizadores são baseados no clássico algoritmo do Gradiente Descendente, que busca usar o valor negativo do gradiente obtido em cada camada para atualizar os pesos desta camada (o negativo do gradiente aponta para a direção de minimização da função de erro). No entanto, esse algoritmo clássico pode ser bem lento, dependendo do treinamento que esteja sendo executado. Assim, surgiram algumas variações que vamos discutir a seguir.

- **Otimizador com Momentum:** adiciona um termo ao gradiente descendente clássico para considerar a velocidade da descida em direção ao valor mínimo de erro: conforme a velocidade da descida aumenta, também aumenta o valor do passo de treinamento. Esse otimizador é bastante útil para ajudar a escapar de mínimos locais durante o treino.
- **RMSProp:** acumula os gradientes de passos anteriores recentes para adicionar um termo envolvendo esse valor na atualização dos pesos das camadas da rede.
- **Adam:** talvez o algoritmo mais popular de otimização usado no treinamento de redes profundas. Faz uma combinação do *Momentum* + *RMSProp*. É um algoritmo de taxa de aprendizado adaptativa e requer ajuste de poucos parâmetros para funcionar

4. PROBLEMA DO SOBREAJUSTE

Como as redes neurais profundas apresentam muitas camadas de neurônios, teremos um número enorme de pesos/parâmetros, o que dá ao modelo uma grande capacidade de modelar funções bem complicadas. Justamente todo esse poder pode fazer com que o modelo comece a aprender o ruído dos dados muito rapidamente e não generalize bem quando aplicado a novos dados.

Tal fenômeno é conhecido como *sobreajuste* dos dados de treinamento. Quanto mais complexo o modelo, maior sua capacidade de aprender esse ruído mais rapidamente e se tornar um generalizador ruim. Desse modo, são necessárias técnicas para tentar impedir o *sobreajuste*, sobretudo em modelos muito profundos.



4.1 Parada Prematura

A técnica mais simples e empregada mesmo em redes neurais mais rasas: observar, durante o treinamento, o desempenho do modelo em um conjunto de validação. E encerrar o treinamento quando o modelo começar a apresentar *performance* ruim nesse conjunto. Para tornar essa técnica mais eficaz, podemos definir um número de passos de treinamento de “paciência”, que são observados para confirmar que a queda no desempenho não foi pontual.

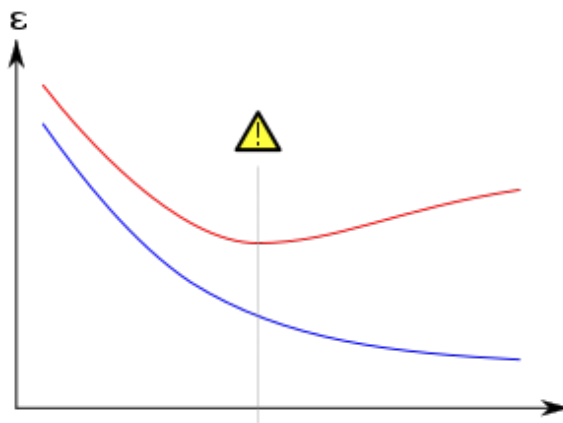


Figura 3: ponto de parada prematura quando o erro de validação começa a subir

4.2 Regularização e Dropout

A regularização restringe os pesos de uma rede neural, diminuindo os graus de liberdade da rede e reduzindo a chance de sobreajuste. Basicamente, adicionamos uma penalidade na função de erro que considera a magnitude dos pesos da rede neural: se o valor dos pesos for muito elevado, a penalização no erro será maior, o que acarretará em uma atualização dos pesos menor. As duas formas de regularização mais famosas, L1 (Lasso) e L2 (Ridge), basicamente se baseiam na soma dos quadrados de cada peso ou dos módulos de cada peso.

Já o Dropout é uma técnica um tanto curiosa, mas que apresenta resultados surpreendentes. A ideia por trás do Dropout é dar a cada neurônio uma probabilidade de ele estar presente ou não na etapa de treinamento (por exemplo, 25% de chance de ser apagado). Assim, a cada passo de treinamento, são sorteados os neurônios que estarão presentes. E os demais, junto de suas conexões, são apagados naquele momento. Assim, os pesos das conexões não sofrerão ajustes nessa etapa.

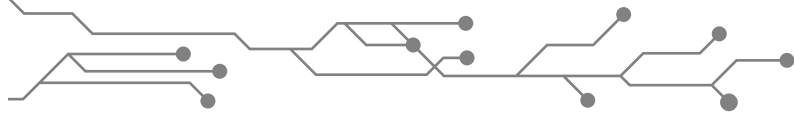


Apesar de não aparentar que vá funcionar em um primeiro momento, essa técnica tem a fundamentação baseada em como os neurônios em nossos cérebros funcionam; ou seja: caso um neurônio morra, os demais deverão aprender a fazer as tarefas pelas quais o morto era responsável, criando uma redundância, de modo que uma tarefa possa ser executada por vários neurônios. Da mesma maneira, como vários neurônios da rede neural não participam de algumas etapas de treinamento, eles devem aprender a adaptar-se às outras etapas de treino, para serem menos suscetíveis a pequenas mudanças nas entradas da rede.

As duas técnicas já estão implementadas em pacotes como TensorFlow e PyTorch. Devem ser aplicadas em modelos profundos, para obter melhores resultados.

4.3 Data Augmentation

Uma forma de reduzir o sobreajuste é fornecer muito mais dados de treino para a rede do que o número de parâmetros que o modelo tem. Dessa forma, será mais difícil para o modelo aprender pontos específicos dos dados de treinamento. Porém, obter um grande volume de dados pode ser uma tarefa cara, demorada e difícil, dependendo do tipo de dados com que estamos trabalhando. Dados de origem da medicina, por exemplo, geralmente têm volume bastante reduzido. Uma das formas de contornar isso é utilizando a técnica conhecida como *Data Augmentation*, que se baseia em fazer alterações nas amostras de treinamento para compor novas amostras.



As alterações feitas pelo *Data Augmentation* normalmente são operações que modificam os dados de treinamento, como operações de rotação e translação de imagens. Existem algumas técnicas específicas para modificação de dados do tipo áudio ou tabuladores, mas a ideia é sempre a mesma: gerar novos dados que sejam similares aos originais, porém, que apresentem alguma modificação que também seja aprendida pela rede. Com isso, a rede também deverá aprender essas operações e corre menos risco de sobreajustar os dados de treino originais.



2. Redes Neurais Convolucionais

As redes neurais convolucionais constituem um tipo muito importante de arquitetura para redes neurais profundas, fazendo uso da operação da convolução para tentar diminuir o número de parâmetros do modelo profundo, permitindo que mais camadas sejam usadas. Isso é possível, porque a operação de convolução possibilita que a rede neural tenha mais *esparsidade* e maior *compartilhamento de parâmetros*. Tais vantagens fazem com que sejam muito empregadas para tarefas complexas, como classificação de imagens.

1. O CÓRTEX VISUAL

Toda a ideia por trás das redes convolucionais vem inspirada em experimentos realizados pelos ganhadores do Prêmio Nobel de Medicina e Fisiologia, David H. Hubel e Tosten Wiesel, que nos deram um maior entendimento sobre o funcionamento do córtex visual. Em resumo, o trabalho desses cientistas mostra que muitos dos neurônios do córtex visual têm um *campo receptivo local*, reagindo apenas a estímulos em uma pequena região do campo visual. Com a sobreposição desses campos locais é que se forma todo o campo visual.



Outro ponto importante apontado por esses pesquisadores é que alguns neurônios reagem apenas a imagens nas linhas horizontais; outros reagem a linhas com diferentes orientações, e outros ainda têm campos receptivos maiores, reagindo a padrões que são combinações dos padrões de nível inferior. Isso, portanto, leva à ideia de que as saídas de neurônios de níveis superiores são baseadas nas saídas dos neurônios vizinhos de nível inferior.

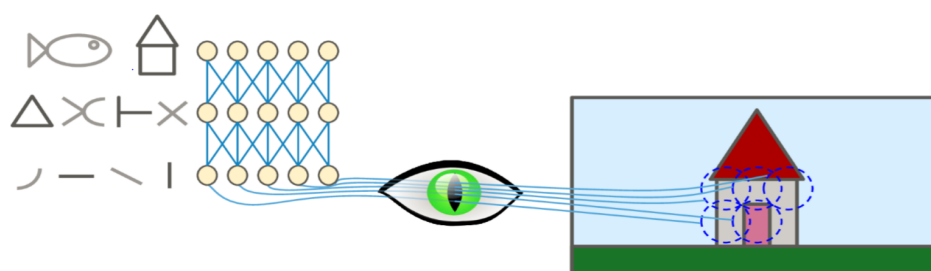


Figura 4: Ilustração dos campos receptivos locais no córtex visual de um felino

2. A CAMADA CONVOLUCIONAL

Os estudos sobre o funcionamento do córtex visual proporcionaram o desenvolvimento de um novo tipo de camada, que seria a base para as redes convolucionais: a camada de convolução. Com esse bloco, temos uma diferença significativa das redes neurais convolucionais para as redes MLPs que já conhecemos: a esparsidade.

Para as MLPs clássicas, todos os neurônios da camada de entrada são conectados a todas as entradas da rede neural. Além disso, todas as saídas desses neurônios são conectadas a todas as entradas da próxima camada da rede neural, gerando um modelo *totalmente conectado*. Nas camadas convolucionais, isso não ocorre; cada neurônio da camada está conectado apenas a um limitado número de entradas dessa camada neural, fazendo um paralelo com os campos receptivos locais que citamos na última seção.

Da mesma forma, uma segunda camada convolucional terá cada um de seus neurônios conectados a um número limitado de saídas da primeira camada convolucional. Isso faz com que as primeiras camadas da rede foquem em características de baixo nível que são reunidas para formar características de nível superior na próxima camada. Essa estrutura hierárquica de características é comum no mundo real e é o que está por trás do sucesso das redes convolucionais.

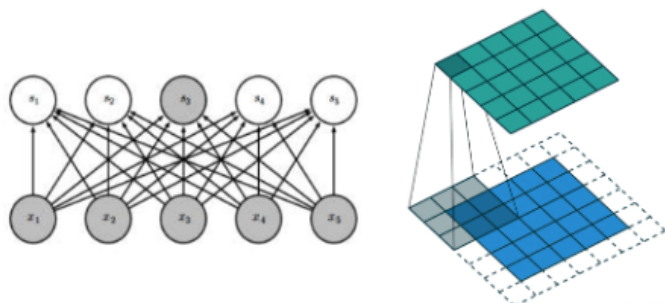


Figura 5: Comparação das conexões de uma MLP clássica (à esquerda) e uma camada convolucional (à direita)

No exemplo da camada convolucional da Figura 5, considerando uma imagem como entrada dessa camada, o primeiro neurônio está conectado apenas aos 4 primeiros pixels da imagem. Uma técnica chamada *padding* é usada, completando algumas posições ao redor da imagem com zeros, para que a saída da camada tenha a mesma dimensão que a entrada. Com esse número mais limitado de conexões, temos menos parâmetros para serem ajustados, e conseguimos uma rede mais *esparsa*.

2.1 Filtros da Camada Convolucional

Como mostrado na figura 5, temos uma pequena “janela” que define quais pixels de uma imagem são conectados a cada neurônio da camada convolucional. No caso específico desse exemplo, temos uma “janela” de tamanho 3x3. Ligado a essa “janela”, temos outro componente muito importante das redes convolucionais: os *filtros* (ou *kernels*). São eles que definem operações aplicadas aos filtros dentro dessa “janela” (que é chamada de tamanho/dimensão do filtro).

Diferentes filtros podem ser aplicados em uma mesma janela, e cada um deles pode ser usado para extrair algum tipo de característica dos campos locais. Se dois filtros são aplicados simultaneamente em um mesmo campo receptivo, dizemos que temos a aplicação de um filtro de dois canais. O resultado da aplicação de um filtro com n canais são n *mapas de características*, que são cópias das imagens originais com aplicação do filtro varrendo todos os campos receptivos dessa imagem (campos esses que são definidos pelo tamanho do filtro).

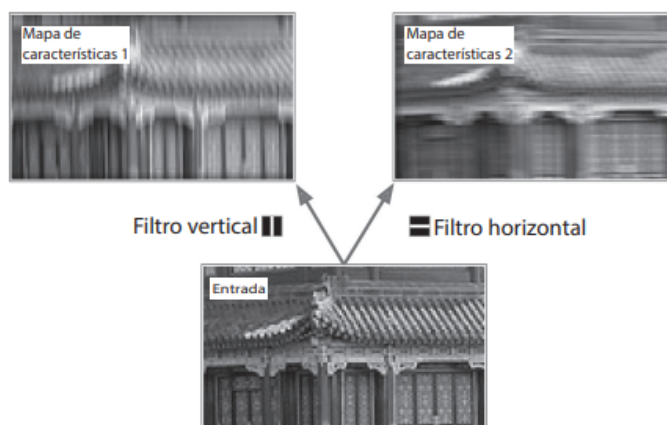


Figura 6: Exemplo da aplicação de dois filtros diferentes (ou um filtro com dois canais).



Os filtros aplicados às entradas da camada convolucional são aprendidos durante o treinamento. O próprio modelo aprende quais características deve extrair dos dados e como combiná-las por meio desse processo. Por exemplo, para identificar uma cruz, seria necessário combinar filtros de linhas verticais e de linhas horizontais.

Além disso, o filtro com n canais aprendido para determinada camada convolucional é o mesmo que é aplicado para todos os campos receptivos locais das entradas daquela camada. Assim faz com que haja um *compartilhamento de parâmetros* (diminuindo consideravelmente a complexidade do modelo final).

3. A CAMADA POOLING

Seu objetivo é subamostrar a saída de uma camada convolucional, de forma que tenhamos o menor custo computacional quando realizando várias operações de convolução (que é a varredura da entrada da camada de convolução usando um filtro com certo número de canais).

Fica mais fácil entender a complexidade da camada de *pooling* pensando no seguinte cenário: considere que a entrada da primeira camada convolucional de uma RNC é uma imagem $256 \times 256 \times 3$ (imagens coloridas possuem 3 canais no padrão RGB), sendo que na primeira camada aplicamos um filtro $3 \times 3 \times 32$, resultando em uma saída $256 \times 256 \times 32$. Na primeira convolução, já multiplicamos o volume do dado de entrada por um fator maior que 10 vezes. Dessa forma, provavelmente haverá outras operações similares e que poderão extrair um número ainda maior de *mapas de características*, tornando o processo de treinamento bastante taxativo, sob o ponto de vista computacional.

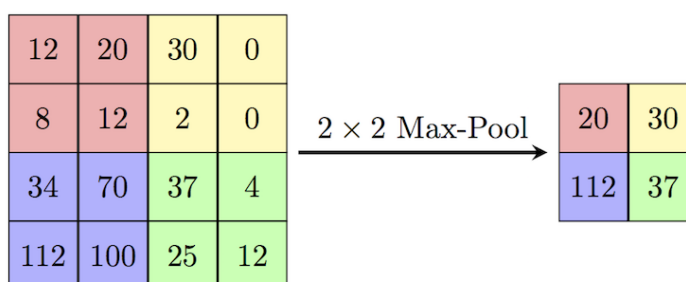


Figura 7: Aplicação da camada Max Pooling com filtro 2×2

Como ilustrado na Figura 7, a camada *pooling* aplica uma subamostragem logo após a operação de convolução, usando uma operação de máximo ou de média. Assim, esse tipo de camada não tem pesos a serem ajustados durante o treinamento, servindo apenas para diminuir o tamanho da saída.

Combinando as camadas convolucional e *pooling*, já temos as ferramentas para construir uma rede convolucional (Figura 8).

Os filtros aplicados em cada camada extraem mapas de características que são subamostrados, resultando em uma coleção cada vez menor de mapas extraídos. Os primeiros mapas extraem características mais simples, que são combinadas para extrair características mais complexas nos mapas das camadas mais profundas. Ao final das camadas convolucionais, temos uma ou duas camadas totalmente conectadas que fazem a classificação, baseando-se nas características encontradas pelas camadas totalmente conectadas.

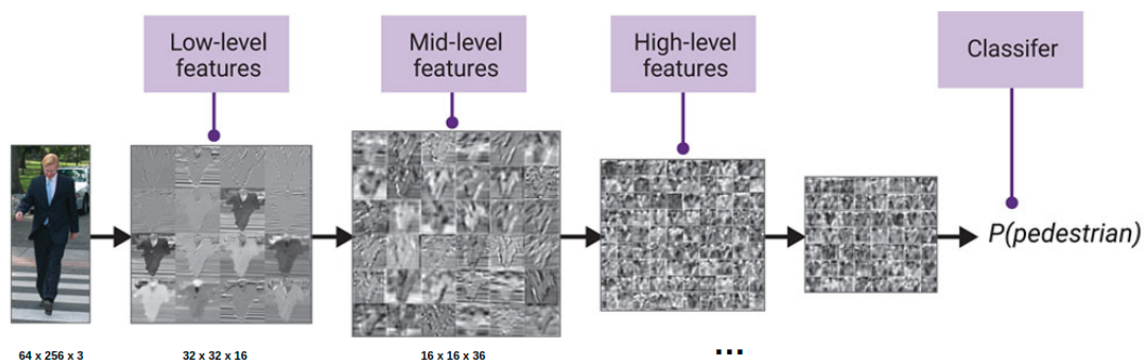


Figura 8: Exemplo de como ocorre a extração de mapas de características em uma RNC

4. PADDING E STRIDE

Dois parâmetros que devem ser escolhidos para o desenvolvimento de uma RNC são o *padding* e o *stride*. São esses parâmetros que definem como será feita a varredura da entrada de uma camada convolucional, influenciando diretamente nos mapas de características, obtidos após a aplicação dessa camada.

Já comentamos sobre o *padding* anteriormente, que é um parâmetro que indica se completaremos os arredores do dado de entrada com zeros, de forma a modificar o tamanho da saída da camada convolucional aplicada. Podemos ter uma saída que seja de mesmo tamanho da entrada ou até maior com a aplicação de *padding*.

Algumas estratégias de *padding* já são implementadas nos principais pacotes de desenvolvimento de modelos convolucionais. Portanto, é necessário definir esse parâmetro para cada camada convolucional da RNC. A estratégia *same* é uma das mais populares, completando a entrada com zeros, de forma que a saída tenha a mesma dimensão da entrada, como ilustrado na Figura 5. No caso de uma imagem como entrada, teremos uma imagem na saída com a mesma altura e largura, podendo ter mais ou menos canais.

O *stride* define como o filtro varrerá toda a entrada da camada convolucional, também influenciando no tamanho dos mapas de características obtidos após a camada. Novamente, para o caso de uma imagem como entrada da camada, se tivermos um *stride* igual a 1, o filtro se deslocará 1 pixel de cada vez para varrer a imagem (aliado ao *padding same*, isso resulta em uma saída de mesmo tamanho da entrada (Figura 5). Caso modifiquemos o valor do *stride* para 2, o filtro terá um passo de 2 pixels durante a varredura, o que pode diminuir o tamanho do mapa em 2 vezes. Dependendo do tamanho do *stride* usado, pode nem ser necessário o uso de uma camada *pooling*, já que a saída da camada será menor do que a entrada.

3. Redes Neurais Recorrentes

As redes recorrentes são talvez tão populares como os modelos convolucionais, muito devido ao tipo de problemas que esse modelo resolve. As redes recorrentes são muito aplicadas em problemas em que a entrada tem alguma dependência temporal, como séries temporais de ações da bolsa de valores, textos com avaliações de clientes e arquivos de áudio de fala humana.

Neste capítulo, iremos explorar mais detalhadamente esses modelos, sua construção e principais diferenças com os modelos vistos nas unidades anteriores.

1. OS NEURÔNIOS RECORRENTES

Nas redes que vimos até o momento, a informação flui em um único sentido, da entrada para a saída da rede. O conceito das redes recorrentes explora justamente uma modificação nos modelos MLP para permitir uma realimentação da informação de saída de uma camada recorrente (Figura 9).

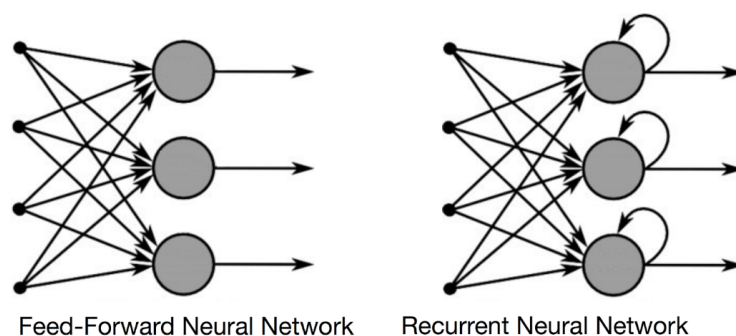


Figura 9: Comparação do fluxo de informação em uma rede feedforward clássica e em uma RNN

Na prática, a recorrência da rede recorrente é alcançada modificando os neurônios que compõem a rede. Assim, cada neurônio de uma rede recorrente, na verdade, é uma coleção de neurônios que recebem a informação de um instante de tempo (ou posição em uma sequência). Eles processam essa informação para retornar uma saída e também alimentam essa saída no neurônio que recebe o próximo instante de tempo. Desse modo, ao invés de termos 1 único neurônio recorrente, temos uma célula recorrente que apresenta n neurônios dentro dela - n o número de instantes de tempo que compõem a entrada. Na Figura 10, temos a visualização de um neurônio/célula recorrente desenrolado, ao longo do tempo, para ilustrar a explicação anterior.

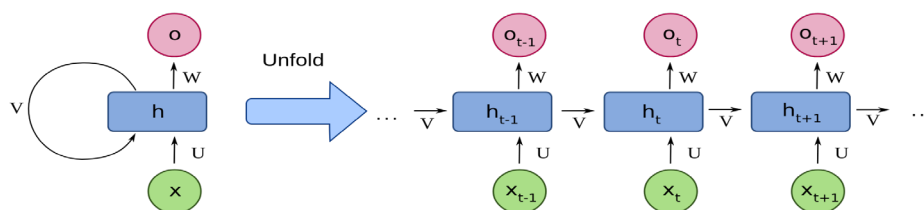


Figura 10: Visualização de um neurônio/célula recorrente desenrolado no tempo

É importante notar que a visualização da Figura 10 é de apenas um neurônio da camada oculta, e que uma camada recorrente pode ter mais de um neurônio/célula recorrente. Além disso, o número de unidades dentro de um neurônio recorrente é igual ao número de passos temporais ou comprimento da sequência que estamos usando como entrada da rede neural. Por exemplo, podemos usar o preço dos últimos 10 dias de uma determinada ação para tentar prever seu preço futuro, o que fará com que neurônios recorrentes contenha 10 unidades.

Uma vez que a representação da rede recorrente desenrolada no tempo da Figura 10 foi entendida, fica mais fácil visualizar como ocorre o treinamento: as saídas obtidas são comparadas com os valores desejados e o erro é retropropagado para atualizar os pesos da rede. Note que agora também temos pesos referentes às conexões entre os neurônios de uma mesma camada. A única diferença é que essa retropropagação ocorre de modo a considerar a ligação entre os neurônios, começando pelo erro do último instante de tempo, que também é propagado para o neurônio correspondente do instante de tempo anterior, e assim por diante.

Podemos pensar que a retropropagação em uma rede recorrente (Figura 10) ocorre *de cima para baixo e da direita para a esquerda*, sendo chamada de *retropropagação através do tempo* (*Backpropagation Through Time - BPTT*). Além disso, assim como nas convolucionais, as redes recorrentes fazem o uso do compartilhamento de parâmetros, com os pesos ligando as entradas, saídas e conexões entre os neurônios, que são os mesmos para todos os instantes de tempo.



2. SEQUÊNCIAS DE ENTRADAS E SAÍDAS

Ao construir uma rede neural recorrente, temos algumas variações de arquitetura que podemos utilizar, dependendo do tipo de saída que queremos que a rede nos apresente:

- **Muitos-para-muitos (*many-to-many*):** a configuração mais óbvia, onde temos números de saídas e entradas maiores que 1. Esse tipo de arquitetura é usado quando temos uma sequência como entrada e queremos também uma sequência na saída da rede (exemplo: provemos um texto na entrada e queremos que a rede gere um texto com significado similar).
- **Muitos-para-um (*many-to-one*):** nessa configuração, alimentamos a rede com uma sequência, mas queremos um único resultado como saída. Essa arquitetura é útil para tarefas como classificação de textos, análise de sentimento e predição de risco baseado na série temporal.
- **Um-para-muitos (*one-to-many*):** aqui nós fornecemos apenas uma entrada para a rede neural e esperamos que ela mapeie essa entrada para uma sequência de saídas. Um exemplo dessa aplicação aparece na tarefa de dar apenas uma imagem para a rede e ela nos fornecer uma descrição dessa entrada.

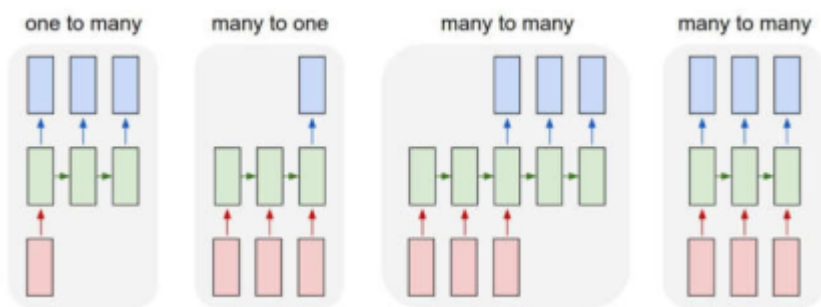


Figura 11: Variações da arquitetura de uma RNR.

Um ponto importante de se ter em mente quando lidamos com redes recorrentes vem no detalhe de que as amostras devem ser sequências com alguma dependência temporal, mas cada ponto da sequência pode ter mais de uma dimensão. Um exemplo seria uma série temporal tridimensional, na qual cada ponto de uma sequência extraída dessa série teria duas dimensões para a entrada da rede recorrente.

3. A CÉLULA LSTM

A unidade/célula *Mémoria Longa de Curto Prazo* (Long Short-Term Memory - LSTM) consiste em um aperfeiçoamento da unidade básica das redes recorrentes, melhorando muito o desempenho desse tipo de modelo. Considerando a célula LSTM como uma caixa preta, ela se comporta exatamente igual a uma célula comum de uma RNR, com a exceção de que a LSTM tem duas entradas que são advindas de saídas do elemento passado (Figura 12). Dizemos que o estado dessa célula está dividido em dois: **h** é o estado de curto prazo e **c** é o estado de longo prazo.

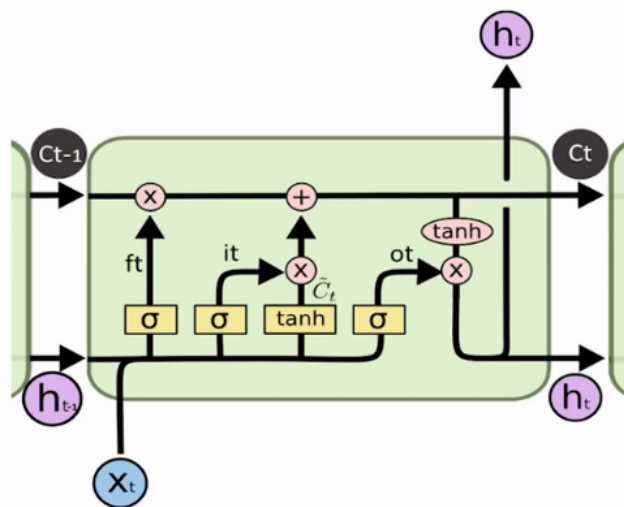


Figura 12: Célula/Unidade LSTM

A ideia por trás da LSTM é que a rede consiga identificar o que manter e o que deve armazenar e o que deve ser descartado do estado **c** advindo da célula anterior. Por meio das multiplicações e adições feitas dentro da célula, novas memórias são adicionadas ao estado **c**(t-1) e outras são descartadas, gerando o estado **c**(t), que é propagado para a próxima célula, sem nenhuma outra transformação adicional. Assim, conforme a informação vai sendo propagada pelas unidades LSTM dentro de um neurônio recorrente, novas memórias são adicionadas ao estado de longo prazo **c** e outras são apagadas. O estado **c**(t) também é copiado e passa por uma função $\tanh$, que é filtrado logo em seguida por uma multiplicação e produz o estado de curto prazo **h**(t). Esse é o correspondente à saída do intervalo de tempo **o**(t), quando multiplicada pelo peso correspondente da saída da célula.

Os processos que ocorrem dentro de uma LSTM podem parecer confusos à primeira vista, mas são bem fundamentados e realmente resultam em uma drástica melhora da *performance* para os modelos RNR. Uma variação mais recente da célula LSTM se apresenta na forma das GRUs (Gated Recurrent Unit), que foram propostas por Kyunghyun Cho e associados, no ano de 2014, como uma versão simplificada da LSTM. Não entraremos em detalhes sobre essa célula aqui, pois as células LSTM ainda são amplamente utilizadas em modelos RNR e o funcionamento dos dois tipos de célula são bem similares.

4. Autoencoders e Transfer Learning

Cobrimos as três principais arquiteturas de aprendizado profundo nos capítulos anteriores. Elas servem como blocos de construção para algumas arquiteturas ainda mais complexas. Veremos uma dessas arquiteturas neste capítulo na forma do *autoencoder*, que basicamente combina duas redes neurais profundas *feedforward* para formar uma rede que consegue criar representações dos dados de entrada. É extremamente útil para tarefas como geração de novos dados e segmentação de imagens médicas.

Ao final deste capítulo, também veremos sobre a técnica *transfer learning* e como ela pode nos ajudar a aproveitar modelos poderosos já treinados para facilitar o treinamento de nossos modelos profundos.

1. O AUTOENCODER

A ideia por trás do *autoencoder* é muito engenhosa: se a cada camada de uma rede neural são extraídas características de ordem cada vez mais elevada, a rede está aprendendo a cada camada a gerar uma representação do dado de entrada. Quanto mais camadas, mais complexa é essa representação. Podemos, então, acoplar à saída desta rede (antes da camada de saída da rede) outra rede, que tem a tarefa de “desenrolar” essa representação para reconstruir o dado original.

Assim, podemos treinar essas duas redes em conjunto, de forma não-supervisionada, para que, quando apresentado um dado na entrada da rede, ela consiga reconstruir o mesmo dado em sua saída. A saída desejada será a própria entrada e o erro será zerado se a rede conseguir gerar exatamente o mesmo dado. A Figura 13 ilustra um *autoencoder*: a primeira rede (responsável por fazer a codificação do dado de entrada em uma *representação* comprimida) se chama *encoder* e a segunda rede (responsável por decodificar a representação para o dado original) é chamada de *decoder*.

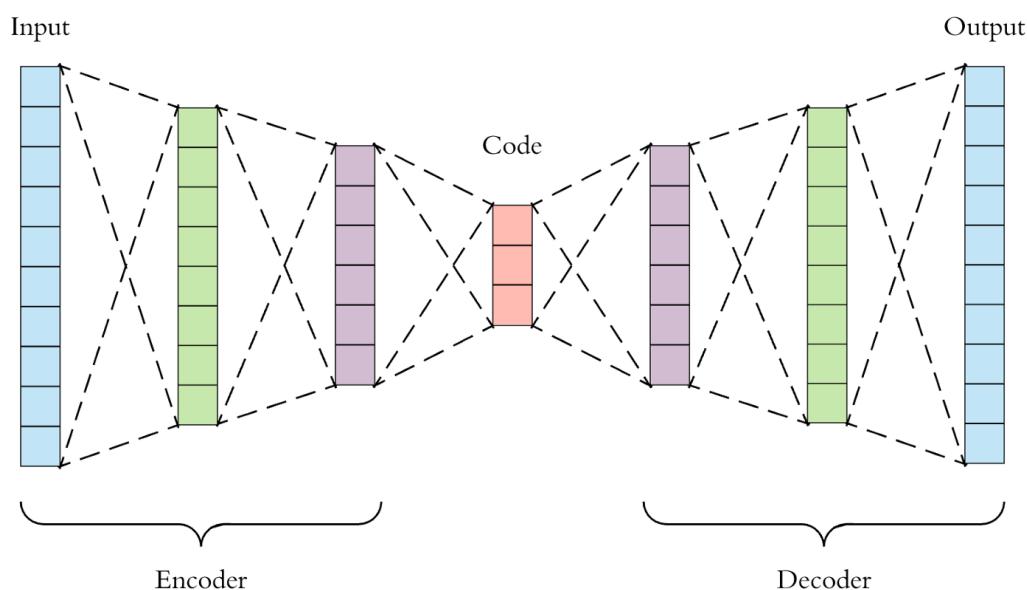


Figura 13: Representação de um autoencoder

À primeira vista, um modelo que apenas gera o mesmo dado de entrada não parece de muita utilidade. No entanto, se separarmos os dois modelos após o treinamento bem-sucedido, temos duas ferramentas bem poderosas: o modelo *encoder* é capaz de detectar características, enquanto o *decoder* é capaz de gerar novos dados a partir de representações geradas de modo aleatório. Assim, o autoencoder aprende durante o treinamento a gerar representações eficientes dos dados de entrada e a decodificar essas representações.

1.2. Arquitetura e treinamento de um Autoencoder

Como representado na Figura 13, um *autoencoder* tem a mesma arquitetura de uma rede MLP, com a única condição de que o número de saídas do *encoder* deve ser igual ao número de entradas do *decoder*. O número de camadas e neurônios em cada um dos dois componentes é hiperparâmetro que deve ser ajustado durante o treinamento para otimizar o resultado do modelo. Mas existe um detalhe: o *autoencoder* é simétrico com relação à camada central, sendo as duas redes que o compõem espelhos.



Como o *autoencoder* reconstrói um dado de entrada a partir de uma representação com menos dimensões (comprimida), ele não pode simplesmente copiar a imagem de entrada. Isso força esse tipo de modelo a encontrar as características mais importantes dos dados de entrada e codificá-las, de modo que possa gerar dados que contenham essas características importantes. Isso também faz com que características que a rede julgue que não sejam importantes para a reconstrução, como ruído em alguns casos, sejam descartadas.

Quando um *autoencoder* tem mais de uma camada para o *encoder* ou o *decoder*, ele é chamado de *autoencoder empilhado*. Quanto mais camadas, mais poderosos os modelos, o que também pode se tornar um problema: se o *autoencoder* ficar muito poderoso, ele pode mapear até mesmo características pouco importantes para a reconstrução do dado, não aprendendo uma representação útil.

Para o treinamento de *autoencoders*, uma estratégia bem eficiente é treinar o *autoencoder* por partes: treinamos *autoencoders* menores (rasos) e depois os empilhamos para formar o modelo final. Isso é feito por fases: primeiramente, treinamos um *autoencoder* para aprender a reconstruir as entradas como discutimos. Após isso, treinamos outro *autoencoder* para reconstruir as saídas da camada central/oculta do primeiro autoencoder. Ao final, empilhamos os dois modelos e construímos o modelo final, como ilustrado na Figura 14.

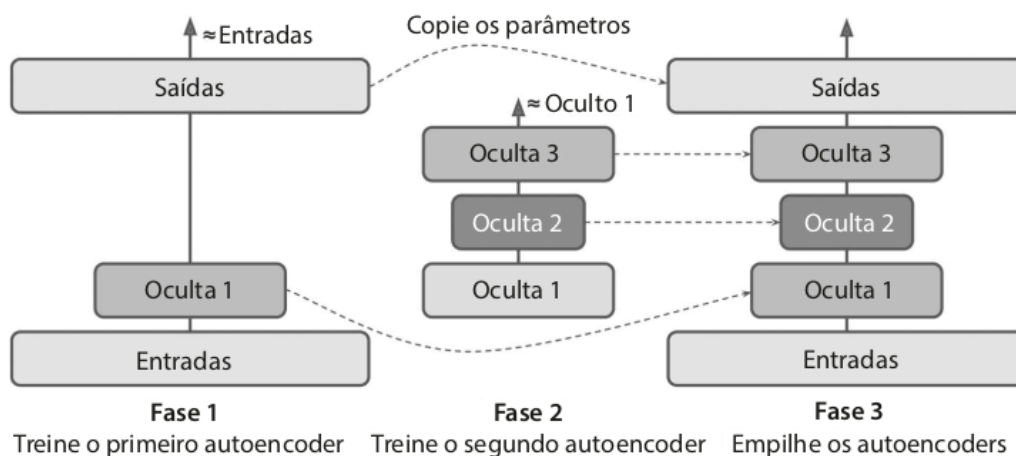


Figura 14: Treinamento de autoencoder profundo por partes

1.3. Características Importantes para a Reconstrução e Aplicações

Além de poder visualizar a saída do modelo para checar o quão bom são as representações geradas pelo *autoencoder*, também podemos observar as características aprendidas usando uma série de técnicas diferentes. Uma dessas técnicas consiste em criar uma representação baseada na ativação dos neurônios do *encoder*. Como exemplo, podemos considerar a primeira camada e criar uma imagem onde cada pixel é correspondente ao peso da conexão entre um pixel da imagem original e o um neurônio dessa camada. Ao plotar essas intensidades de pesos baseados nas posições de pixels da imagem, podemos obter representações como as mostradas na Figura 15, ilustrando regiões importantes para a codificação.

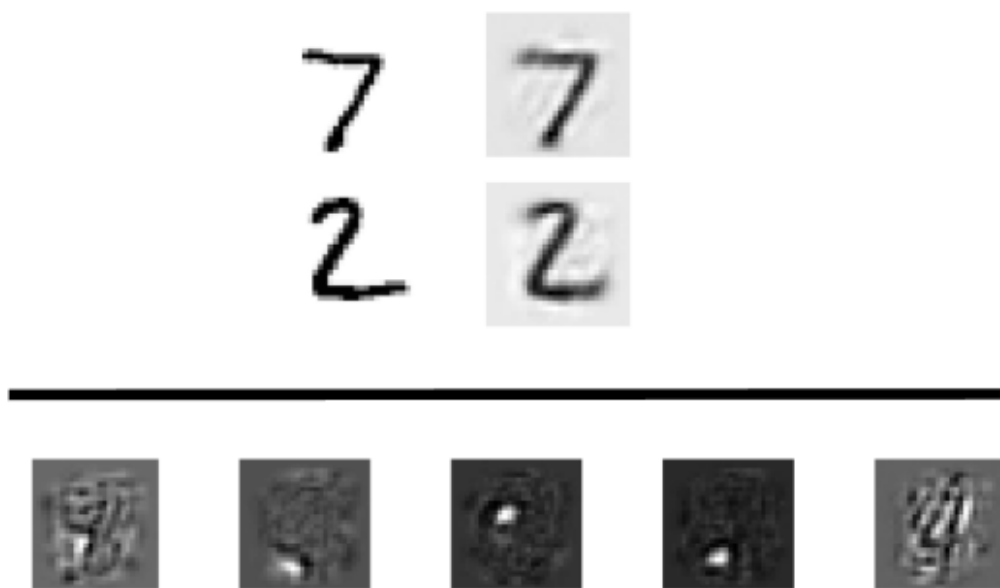


Figura 15: Topo - Imagens reais (esquerda) e imagens reconstruídas (direita) pelo *autoencoder*. Abaixo - visualização de características aprendidas pelos neurônios da primeira camada do mesmo *autoencoder*

Quanto às aplicações, temos uma vasta gama de tarefas em que os *autoencoders* são empregados. Podemos utilizar os *autoencoders* para o pré-treinamento de um modelo profundo de uma forma não-supervisionada, de modo que o *encoder* aprenda a extrair características importantes, para depois receber um ajuste fino para realizar alguma tarefa de classificação ou regressão. Também podemos usar os *autoencoders* para a remoção de ruído em dados: adicionamos ruído às entradas que apresentamos ao *autoencoder*, só que treinando o modelo para que ele gere as imagens originais sem esse ruído. Na entrada do modelo teríamos o dado + ruído, mas a saída desejada é o dado sem o ruído.

Assim, o modelo será forçado a remover o ruído, podendo ser aplicado a outros dados ruidosos para fazer essa filtragem. Uma variação conhecida como *autoencoders variacionais* são especializados para a geração de dados, tendo um componente probabilístico que atua mesmo após o treinamento para gerar esses novos dados. Um tipo de rede bastante usado para a segmentação de imagens médicas é a U-Net, que é basicamente um *autoencoder* composto por redes convolucionais para o *encoder* e *decoder*.

2. TRANSFER LEARNING

Citamos em outros capítulos a técnica *transfer learning* como uma alternativa para minimizar o tempo e custo computacional para treinar modelos muito profundos. Essa técnica simples se baseia na reutilização de modelos já treinados para tarefas similares. Se tenho uma tarefa de classificação de imagens de plantas, posso tentar reutilizar modelos treinados para classificar

fotos de objetos, apenas fazendo um ajuste fino neste modelo. Principalmente para tarefas em que a quantidade de dados é escassa, o *transfer learning* pode viabilizar a utilização de modelos profundos que seriam impossíveis de treinar do zero, devido ao grande número de parâmetros para serem ajustados com poucos dados.

Talvez a principal aplicação da técnica seja em tarefas de classificação de imagens, nas quais muitas tarefas não permitem que tenhamos um banco de imagens muito grande para o treinamento dos nossos modelos. Por exemplo, para classificação de doenças em plantas a partir de imagens. Para esses casos, podemos utilizar algum modelo treinado em bancos de dados enormes, como o *Imagenet* do Google, que é um dataset com mais de 14 milhões de imagens com diversas classes, como cachorro, gato, carro, barco, flor, etc.

Para o caso de imagens especificamente, já sabemos que os melhores modelos se baseiam em redes convolucionais. Existem diversas arquiteturas de redes convolucionais treinadas e otimizadas baseadas em dados da *Imagenet*. A arquitetura VGG16, por exemplo, já registrou ótimos resultados para a classificação de imagens de forma correta, e uma das mais populares arquiteturas de redes convolucionais. Podemos reutilizar a maior parte das camadas de uma rede VGG16 treinada com dados da *Imagenet*, descartando apenas as camadas totalmente conectadas ao final da rede. Ao adicionar novas camadas totalmente conectadas, estamos aproveitando a capacidade de extrair características de imagens aprendida pela VGG16 no *Imagenet*, transferindo essa capacidade para outro modelo, como o de classificação de doenças em plantas.

Uma vez que conectamos novas camadas ao final de uma rede pré-treinada e realizamos o treinamento com novos dados, estamos fazendo com que as últimas camadas aprendam a combinar as características que são extraídas pelas camadas pré-treinadas para realizar a nova tarefa. No entanto, é necessário “congelar” os pesos das camadas pré-treinadas, impedindo que sejam modificados durante o novo treinamento (chamado de ajuste fino), fazendo a rede esquecer a extração de características aprendida durante o treinamento prévio.

Terminado esse novo treinamento e obtendo resultados satisfatórios, podemos “descongelar” os pesos das camadas pré-treinadas e treinar o modelo como um todo durante algumas poucas épocas, de modo a finalizar o ajuste do modelo à nova tarefa. Esse processo é ilustrado na Figura 16.

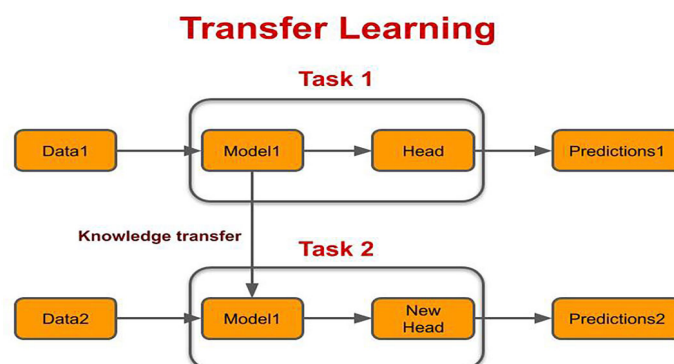
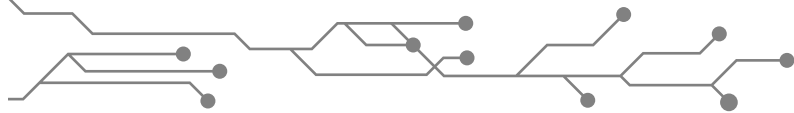


Figura 16: Esquema de treinamento usando *transfer learning*

Apesar do *transfer learning* ser muito usado para tarefas envolvendo imagens (devido aos datasets enormes disponíveis, como *Imagenet*, *CIFAR*, etc.), nada impede que isso seja usado com redes profundas *feedforward* ou RNRs. Somente temos que conseguir um modelo pré-treinado em uma tarefa similar à que temos em mãos. Também temos a opção de pré-treinar o modelo em uma forma não supervisionada usando *autoencoders*, usando exemplares com *labels* em um segundo momento do treinamento.





Coordenadoria de
Educação Aberta e a Distância